ENLANG FOR DEVELOPERS

The Complete Master Specification & Release Guide (v2.0)
Author & Architect: Spandan Prayas Patra

The Complete Master Specification & Release Guide (v2.0)

Author & Architect: Spandan Prayas Patra

Version: 2.0.0 — Enterprise Release Edition

Publisher: Universal EnLang Foundation Press

> *"Programming should be as clear, expressive, and frictionless as human thought. EnLang makes natural English a first-class, production-grade programming language across the entire software stack."*

> — **Spandan Prayas Patra**

TABLE OF CONTENTS

21. **Chapter 21: Complete Production Case Study — Lumina Workspace**

22. **Appendix A: Universal Natural Syntax Matrix**

23. **Appendix B: CLI & EnLang Package Manager (EPM) Operations Manual**

CHAPTER 1: PHILOSOPHICAL ARCHITECTURE & TRANSPILATION CORE

For over seven decades, software engineering has forced developers to think in artificial syntax and symbols (`{}`, `;`, `=>`, `::`). EnLang was designed by **Spandan Prayas Patra** to invert this paradigm: to allow developers to express application logic in pure, natural English while compiling deterministically into 1:1 clean, native target languages.

EnLang is not an LLM wrapper. It is a **multi-target compiler engine**:

- `.enlg` ➔ Transpiles to Python 3

- `.enlgf` ➔ Transpiles to HTML5

- `.enlgd` ➔ Transpiles to CSS3

- `.enlgs` ➔ Transpiles to JavaScript ES6+

- `.enlgdb` ➔ Transpiles to SQL

CHAPTER 2: VARIABLES, OPTIONAL TYPES & VALUE EXPRESSIONS

Variable declarations in EnLang support expressive natural phrasing. Type annotations are completely optional.

```
# Natural Variable Assignments
set score to 100
let score = 100
store 100 in score

# Optional Type Annotations (v2.0)
define number age as 20
define decimal price as 99.50
define text username as "Spandan"
define boolean isActive as true
define list fruits as ["Apple", "Banana"]
define dictionary profile as {"name": "Spandan"}
```

CHAPTER 3: OUTPUT & DISPLAY TAXONOMY

EnLang provides 4 natural synonyms for printing output to the terminal:

```
display "Welcome to EnLang v2.0"   # Primary keyword
print "Processing record..."         # Direct alias
show "Status: Active"               # Direct alias
output "Calculation Complete"       # Direct alias
```

CHAPTER 4: THE 3-LEVEL NATIVE INTERACTIVITY ARCHITECTURE

EnLang features a 3-level tier for combining natural code with raw target code:

```
Level 1: Pure EnLang (Default)         ➔ set area to width times height
Level 2: Inline Native Marker          ➔ set val to @python(math.sqrt(144))
Level 3: Multi-Line Native Block       ➔ python: ... end python
```

Level 2 & 3 Examples:

```
set root to @python(math.sqrt(25))

python:
import sys
print("Executing in native Python runtime version:", sys.version)
end python
```

CHAPTER 5: COMPREHENSIVE LOOP TAXONOMY

EnLang provides 5 distinct loop constructs to handle every iteration scenario:

5.1 Repeat Count Loop (`repeat N times do:`)

```
repeat 5 times do:
    display "Processing batch item..."
```

5.2 For-Each & Direct For Loops

```
# Natural English For-Each
for each fruit in ["Apple", "Banana", "Cherry"] do:
    display fruit

# Direct For Loop
for item in ["Apple", "Banana", "Cherry"]:
    display item
```

5.3 While Conditional Loops

```
# Direct While Loop
set i to 1
while i <= 5:
    display i
    increment i by 1

# Natural English While Loop
set count to 1
while count is less than or equal to 5 do:
    display count
    increment count by 1
```

5.4 Loop Control Statements (`break` & `continue`)

```
for each num in [1, 2, 3, 4, 5] do:
    if num is equal to 2 then:
        continue     # Skip 2
    if num is equal to 4 then:
        break        # Terminate loop at 4
    display num
```

CHAPTER 6: FUNCTION & RECURSION MASTERY

EnLang supports both standard function signatures and expressive natural English declarations:

6.1 Standard Function Signature

```
function print_numbers(n):
    if n is greater than 10 then:
        return
    display n
    print_numbers(n plus 1)

print_numbers(1)
```

6.2 Expressive Natural English Function Signature

```
function numbers using n:
    if n is greater than 10 then:
        return
    display n
    call numbers with (n plus 1)

start numbers from 1
```

Supported Natural Phrasings:

- Function Headers: **`function foo using n:`**, **`function foo taking n:`**, **`action foo given n:`**, **`task foo for n:`**

- Function Calls: **`start foo from 1`**, **`start foo with 1`**, **`call foo with 1`**, **`run foo using 1`**

CHAPTER 7: PATTERN MATCHING (`match / case / default`)

```
set role to "admin"

match role:
    case "admin":
        display "Full System Access Granted"
    case "editor", "author":
        display "Content Modification Access Granted"
    case is greater than 50:
        display "Score Requirement Met"
    default:
        display "Guest Access Only"
end match
```

CHAPTER 8: EXCEPTION HANDLING & ERROR CONTROL

```
try:
    set result to 100 divided by 0
except:
    display "Runtime exception caught safely"
finally:
    display "Cleanup block executed"

raise ValueError with message "Input out of bounds"
throw error "Authentication failure"
```

CHAPTER 11: FILE I/O, SYSTEM OPERATIONS & HASHING SECURITY

```
# File Operations
write "Initial System Log Data" to file "system.log"
read file "system.log" and store in log_content

# Security & Hashing
hash "SecretPassword123" with sha256 and store in hashed_pass
display "Hashed Signature: " plus hashed_pass

# Environment Variables
get environment variable "PATH" and store in sys_path
check if path "system.log" exists and store in file_exists
```

CHAPTER 12: FRONTEND STRUCTURAL MARKUP (`.enlgf` → HTML5)

```
page title "Lumina Portal"

create header with class "top-bar":
    create nav with class "navbar":
        create h1 with text "Lumina Platform"
        create a with href "#dashboard" with text "Dashboard"
    close nav
close header

create main with class "container":
    create hero with title "Welcome Developer", subtitle "Powered by EnLang Multi-Target Engine"
    create button named actionBtn with text "Get Started"
close main
```

CHAPTER 13: STYLING & DESIGN SYSTEMS (`.enlgd` → CSS3)

```
define theme darkTheme:
    primary: "#4338ca"
    background: "#0f172a"
    text: "#f8fafc"
end theme

style header:
    background-color: "#1e1b4b"
    padding: "20px"

style ".container":
    max-width: "1200px"
    margin: "0 auto"

style button:
    background-color: "#4338ca"
    color: "#ffffff"
    border-radius: "8px"
    padding: "10px 20px"
```

CHAPTER 14: CLIENT-SIDE SCRIPTING (`.enlgs` → JAVASCRIPT ES6+)

```
# Client DOM Manipulation
log "Initializing Client App..."

on click "actionBtn" do:
    display "Button Clicked!"
    set innerHTML of "hero" to "Action Activated"

async function fetch_data():
    set res to await fetch("https://api.example.com/data")
    set json_data to await res.json()
    log json_data
```

CHAPTER 15: DATABASE SCHEMAS & QUERIES (`.enlgdb` → SQL)

```
connect to database "app.db" as main_db

define table users with columns id INTEGER PRIMARY KEY, username TEXT, role TEXT

insert record into users with values 1, "Spandan", "Architect"

execute sql "SELECT * FROM users WHERE role = 'Architect'" and store in admin_users
```

CHAPTER 16: NATIVE NLP & AI PRIMITIVES

```
set customer_review to "EnLang compiler speed and syntax are absolutely incredible!"

analyze sentiment of customer_review and store in review_sentiment
```

```
display "Sentiment Score: " plus str(review_sentiment)

extract keywords from customer_review into key_words
display "Extracted Keywords: " plus str(key_words)
```

CHAPTER 17: ENLANG HTTP WEB SERVER & MULTI-FILE ARCHITECTURE

```
# Main Server Entrypoint: server.enlg
import module json

set app_name to "Lumina Web Platform"
set port_num to 8000

display "Booting " plus app_name plus " on port " plus str(port_num)

start web server on port port_num
```

CHAPTER 18: DEVELOPER TOOLING — STATIC SYNTAX CHECKER (`enlang check`)

The **`enlang check`** command performs static analysis on EnLang source files without executing them.

```
enlang check main.enlg
```

Linter Diagnostic Output:

```
================================================================
  EnLang Syntax Checker & Linter — bad_syntax_test.enlg
================================================================
  [ERROR] Line 3: Unclosed string literal detected. (Suggestion: Ensure string quotes are closed prop
erly)
  [ERROR] Line 4: Block header missing trailing colon ':'. (Suggestion: Add a colon ':' at the end of
 the line)
  [ERROR] Line 4: Unsupported natural phrase detected in statement. (Suggestion: Use 'is greater than
' instead of 'is bigger than')
================================================================
  Result: 3 Error(s), 0 Warning(s)
================================================================
```

CHAPTER 19: DEVELOPER TOOLING — INTERACTIVE CLI DEBUGGER (`enlang debug`)

The **`enlang debug`** command launches an interactive step-by-step debugger:

```
enlang debug app.enlg
```

Debugger Commands Matrix:

- **`s`** / **`step`**: Step to next line of EnLang code

- **`c`** / **`continue`**: Resume execution until next breakpoint or termination

- **`v`** / **`vars`**: Display live variable table

- **`b <N>`** / **`break <N>`**: Set breakpoint at line number N

- **`e <expr>`** / **`eval <expr>`**: Evaluate expression in live frame

- **`q`** / **`quit`**: Exit debugger session

CHAPTER 20: CANONICAL GRAMMAR RULES, ORDER & SYNTAX LAYOUT SPECIFICATION

1. **4-Space Indentation Rule**: All indented blocks (**`if`**, **`for`**, **`while`**, **`function`**, **`class`**, **`match`**, **`try`**) MUST use 4 spaces per level.

2. **Trailing Colon Header Rule**: All block headers MUST terminate with **`:`**.

3. **Block Closure Alignment**: **`end match`** and **`end interface`** MUST align with opening indentation level.

CHAPTER 21: PRODUCTION CASE STUDY — LUMINA WORKSPACE

`app.enlg`

```
import module json

display "Booting Lumina Production Node..."

define number active_connections as 100
set status to "OPERATIONAL"

display "System Status: " plus status plus " | Connections: " plus str(active_connections)

start web server on port 8000
```

APPENDIX A: UNIVERSAL NATURAL SYNTAX MATRIX

| Category | EnLang Natural Syntax | Native Target Output |

| :--- | :--- | :--- |

| **Output** | **`display x`** / **`print x`** / **`show x`** / **`output x`** | **`print(x)`** |

| **Variables** | **`set x to 10`** / **`store 10 in x`** | **`x = 10`** |

| **Function Def** | **`function foo using n:`** / **`action foo given n:`** | **`def foo(n):`** |

| **Function Call**| **`start foo from 1`** / **`call foo with 1`** | **`foo(1)`** |

| **Repeat Loop** | **`repeat 5 times do:`** | **`for _ in range(5):`** |

| **For Loop** | **`for each x in list do:`** / **`for x in list:`** | **`for x in list:`** |

| **While Loop** | `while x <= 5:` / `while x is less than 5 do:` | `while x <= 5:` |

| **Native Escape**| `@python(math.sqrt(25))` | `math.sqrt(25)` |

APPENDIX B: CLI & PACKAGE MANAGER (EPM) OPERATIONS MANUAL

```
enlang run app.enlg           # Compiles & executes backend logic
enlang check app.enlg         # Performs static analysis & linting
enlang debug app.enlg         # Launches interactive step-by-step debugger
enlang build index.enlgf      # Transpiles source into native target file
enlang server --port 8000     # Launches zero-config HTTP host server
epm init                      # Initializes new EnLang package
epm add py:requests           # Installs Python PyPI dependency
epm add web:chart.js          # Installs Web NPM dependency
```
